Name: Rameen Javed

Course: Digital Image Processing

Instructor: Mr. Ghulam Ali

Report

Project: Smart Image Enhancement & Analysis System

Contents

1. Objective 3
2. Methodology 3
3.Phase-wise Implementation 3
3.1 Image Acquisition & Understanding 3
3.2 Sampling & Quantization (Lab 02) 4
Sampling: 4
Quantization: 4
Analysis 8
3.3 Geometric Transformations 8
3.4 Intensity Transformations 11
3.5 Histogram Processing 13
3.6 Final Integrated System 15
4. Results 15
5. Conclusion 19
6. Questions & Answers 19
7. Tools Used 20

1. Objective

The objective of this project is to design and implement a complete image enhancement system using MATLAB by applying various image processing techniques including sampling, quantization, transformations, intensity adjustments, and histogram processing.

2. Methodology

The system follows a structured pipeline:

Input Image → Analysis → Transformation → Enhancement → Output

3.Phase-wise Implementation

3.1 Image Acquisition & Understanding

- The input image was loaded using MATLAB.
- It was converted from RGB to grayscale.
- Image properties such as resolution, data type, and partial pixel values were displayed.

Observation:
The grayscale image simplifies processing by reducing complexity while preserving essential information.

```
img = imread('/MATLAB Drive/hawkes_bay_in.jpg');   % put your image in same folder
figure, imshow(img), title('Original Image');

%% Convert to Grayscale
if size(img,3) == 3
    gray = rgb2gray(img);
else
    gray = img;
end
figure, imshow(gray), title('Grayscale Image');

%% Display Info
disp('Image Info:');
disp(size(gray));
disp(class(gray));
disp(gray(1:5,1:5));   % partial matrix
```

```
Command Window
Image Info:
   225   225

uint8
   130   130   131   131   131
   130   131   131   131   132
   131   131   131   132   132
   131   131   132   132   132
   132   132   132   133   133
```

3.2 Sampling & Quantization (Lab 02)

Sampling:

- Image was resized using scales: 0.25, 0.5, 1, 1.5, and 2.

Quantization:

- Bit depth reduced to:
  - 8-bit (256 levels)
  - 4-bit (16 levels)
  - 2-bit (4 levels)

Observation:

- Down-sampling reduces resolution and causes blurring.
- Up-sampling increases size but introduces pixelation.
- Lower bit depth reduces image quality and causes banding.

Comparison table:

Scale / Bit Depth	Resolution	Quality
0.25x	Very Low	Very Blurry, details lost
0.5x	Low	Slight blur
1x (Original)	Original	Good
1.5x	High	Slight pixelation
2x	Very High	Blurry + stretched

```
%% 2. Sampling (Resize)
scales = [0.5, 0.25, 1.5, 2];
for i = 1:length(scales)
    resized = imresize(gray, scales(i));
    figure, imshow(resized), title(['Scale = ', num2str(scales(i))]);
end
```

```
%% Quantization (Bit Depth Reduction)
levels = [256, 16, 4];  % 8-bit, 4-bit, 2-bit
for i = 1:length(levels)
    q_img = floor(double(gray)/(256/levels(i))) * (256/levels(i));
    q_img = uint8(q_img);
    figure, imshow(q_img), title(['Levels = ', num2str(levels(i))]);
end
```

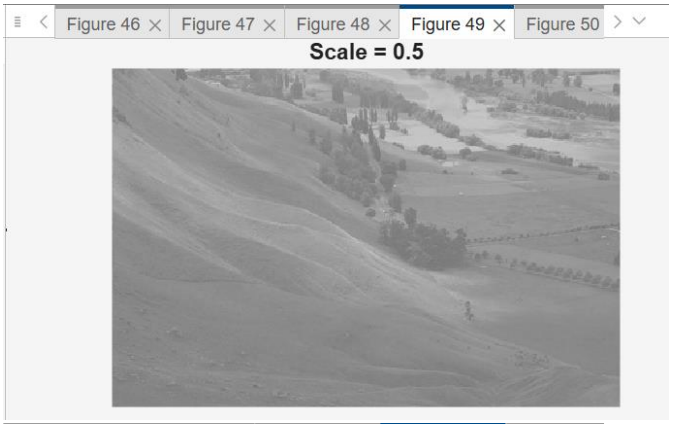


Figure 46
Figure 47
Figure 48
Figure 49
Figure 50
Scale = 0.5

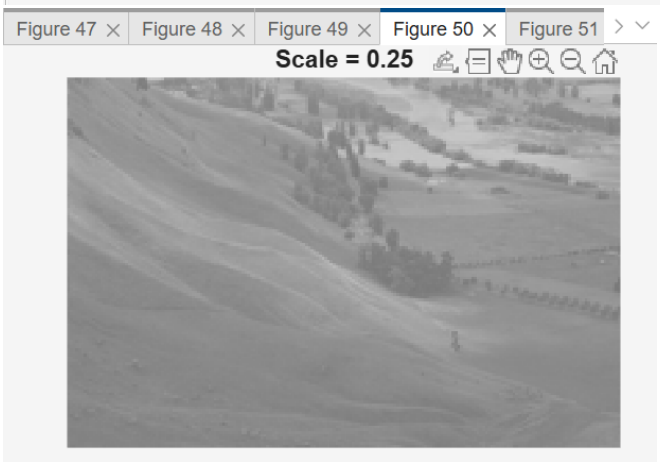
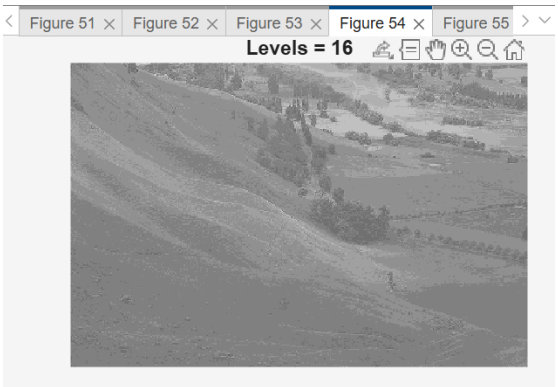


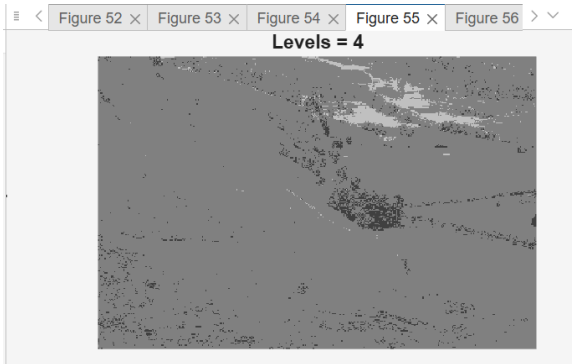
Figure 47
Figure 48
Figure 49
Figure 50
Figure 51
Scale = 0.25

Scale = 1.5

Quantization:

Bit Depth	Levels	Quality
8-bit	256	Best quality
4-bit	16	Noticeable banding
2-bit	4	Very poor, heavy distortion





Analysis

Sampling Analysis:
When the image is down-sampled (0.25, 0.5), resolution decreases and important details are lost, making the image blurry. When up-sampled (1.5, 2), the image size increases but no new information is added, resulting in pixelation and reduced clarity.

Quantization Analysis:
Reducing bit depth decreases the number of intensity levels. At 8-bit, the image is clear. At 4-bit, intensity banding appears. At 2-bit, severe quality degradation occurs due to loss of grayscale detail.

3.3 Geometric Transformations

- Rotation applied at multiple angles (30° to 180°).
- Translation shifted image position.
- Shearing distorted image shape.
- Inverse transformation attempted to restore original image.

Observation:

- Rotation preserves structure but changes orientation.
- Translation is reversible without loss.
- Shearing distorts spatial structure.
- Inverse transformation works when no data is lost.

Analysis

Rotation changes image orientation but preserves structure.

- Translation shifts image position without distortion.
- Shearing distorts shape by slanting the image.
- Inverse transformation can restore the image if no data is lost (e.g., translation is reversible).

```
%% 3. Geometric Transformations

% Rotation
angles = [30, 45, 90, 180];
for i = 1:length(angles)
    rot = imrotate(gray, angles(i), 'bilinear', 'crop');
    figure, imshow(rot), title(['Rotation ', num2str(angles(i))]);
end

% Translation
tform = affine2d([1 0 0; 0 1 0; 50 30 1]);
translated = imwarp(gray, tform);
figure, imshow(translated), title('Translated');

% Shearing
shear = affine2d([1 0.5 0; 0 1 0; 0 0 1]);
sheared = imwarp(gray, shear);
figure, imshow(sheared), title('Sheared');
```

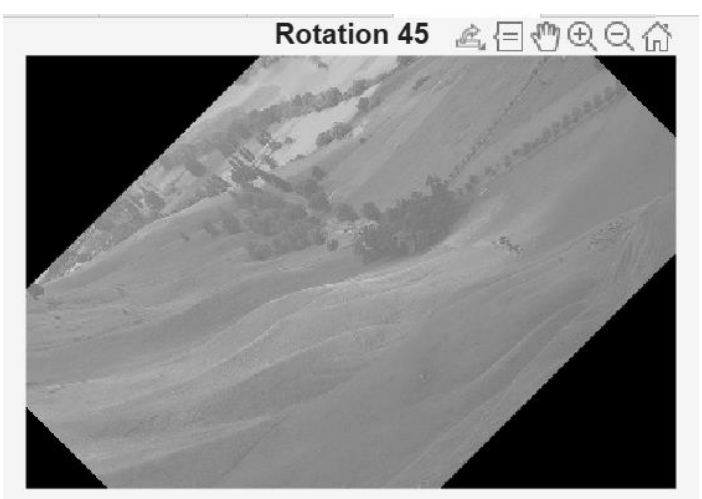
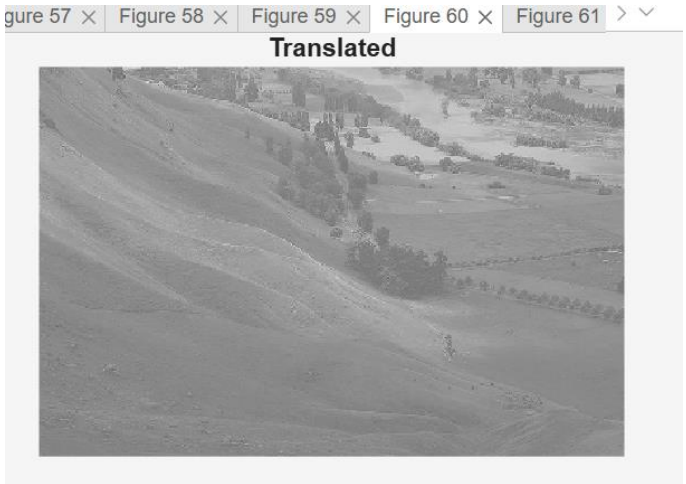
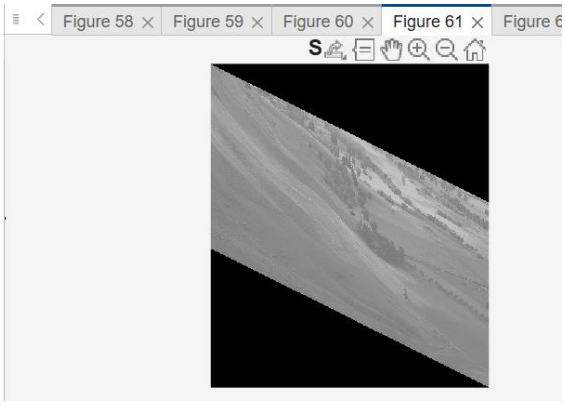


Figure 55
Figure 56
Figure 57
Figure 58
Figure 59
Rotation 90

Figure 56
Figure 57
Figure 58
Figure 59
Figure 60
Rotation 180


gure 57
Figure 58
Figure 59
Figure 60
Figure 61
Translated



3.4 Intensity Transformations

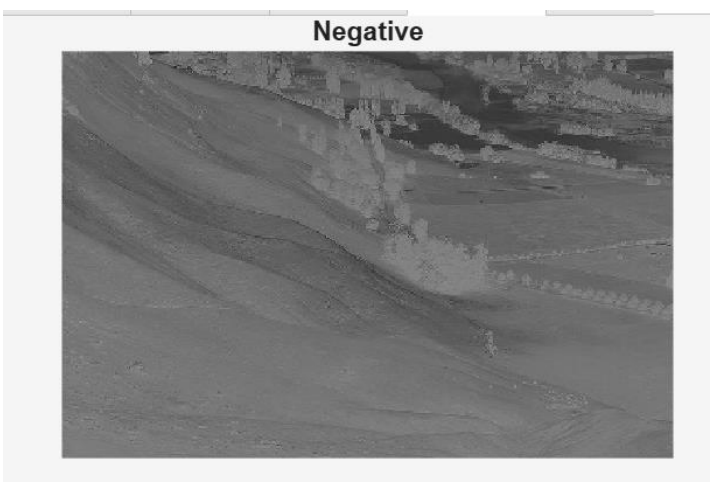
- Negative transformation applied.
- Log transformation used to enhance dark areas.
- Gamma correction applied (γ = 0.5 and γ = 1.5).

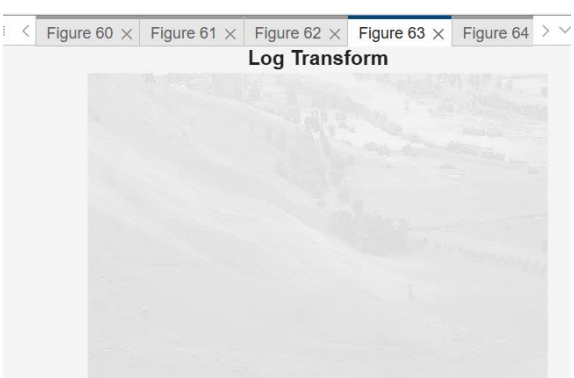
Observation:

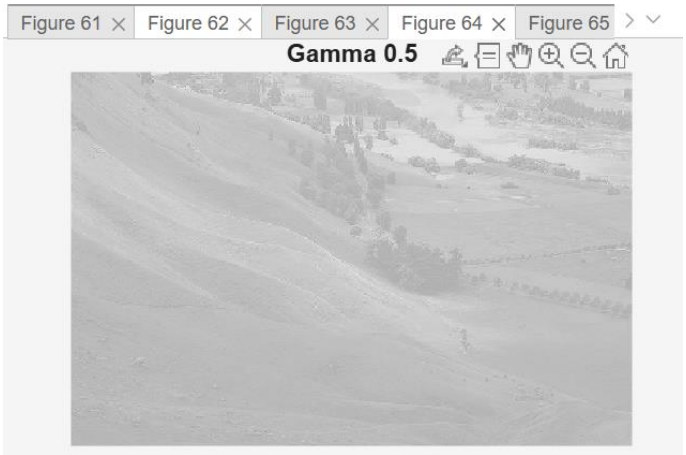
- Negative highlights bright regions.
- Log transformation enhances dark pixels.
- Gamma < 1 brightens image (best for dark images).
- Gamma > 1 darkens image.

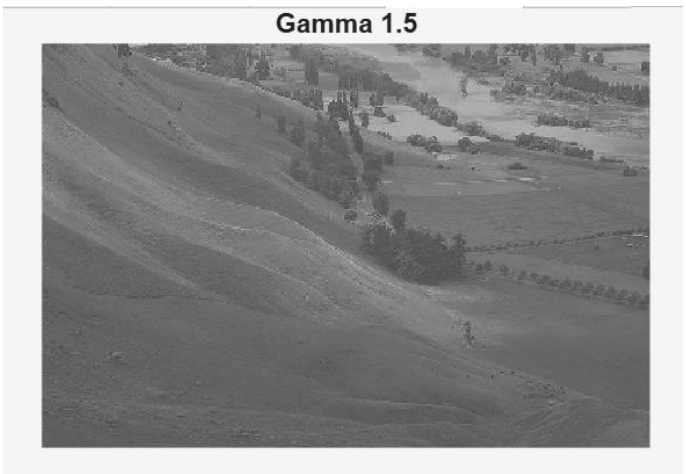
Best method:
Gamma = 0.5 for brightening
Log for highlighting dark details

```
%% 4. Intensity Transformations

% Negative
neg = 255 - gray;
figure, imshow(neg), title('Negative');

% Log Transform
log_img = uint8(255 * log(1 + double(gray)) / log(256));
figure, imshow(log_img), title('Log Transform');

% Gamma Correction
gamma1 = imadjust(gray, [], [], 0.5);
gamma2 = imadjust(gray, [], [], 1.5);

figure, imshow(gamma1), title('Gamma 0.5');
figure, imshow(gamma2), title('Gamma 1.5');
```









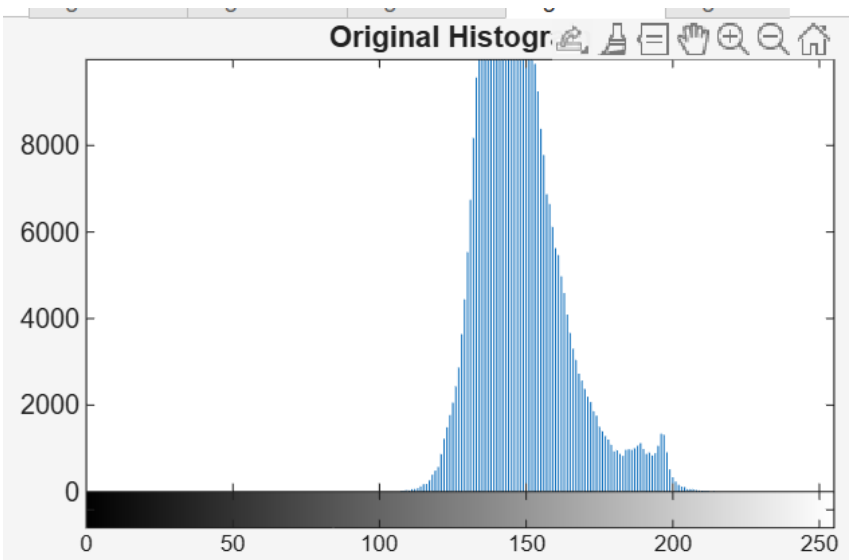
3.5 Histogram Processing

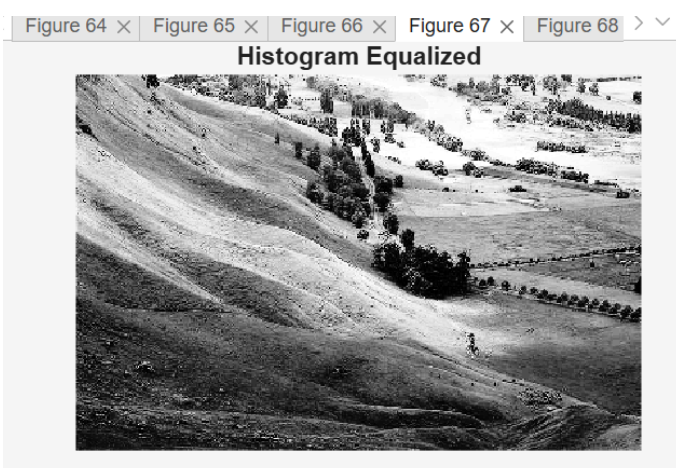
- Histogram of original image plotted.
- Histogram equalization applied.

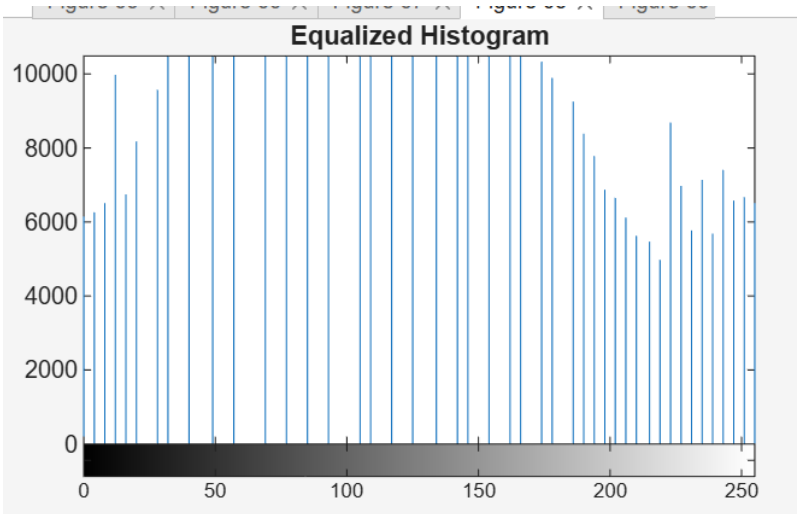
Observation:

- Original histogram showed limited intensity range.
- Equalization spread intensities evenly.
- Contrast improved significantly after equalization.

```
%% 5. Histogram Processing

figure;
imhist(gray);
title('Original Histogram');

eq_img = histeq(gray);

figure, imshow(eq_img), title('Histogram Equalized');
figure;
imhist(eq_img);
title('Equalized Histogram');
```







3.6 Final Integrated System

A final enhancement pipeline was developed:

- Gamma correction (γ = 0.5)
- Histogram equalization

Result:

The final image showed improved brightness, contrast, and overall visual quality.

4. Results

- Enhanced image is clearer and more detailed.
- Noise and low contrast issues are reduced.
- System successfully improves image quality.

Main.m:

```
img = imread('/MATLAB Drive/grains.jfif');   % put your image in same folder
figure, imshow(img), title('Original Image');


%% Convert to Grayscale
```

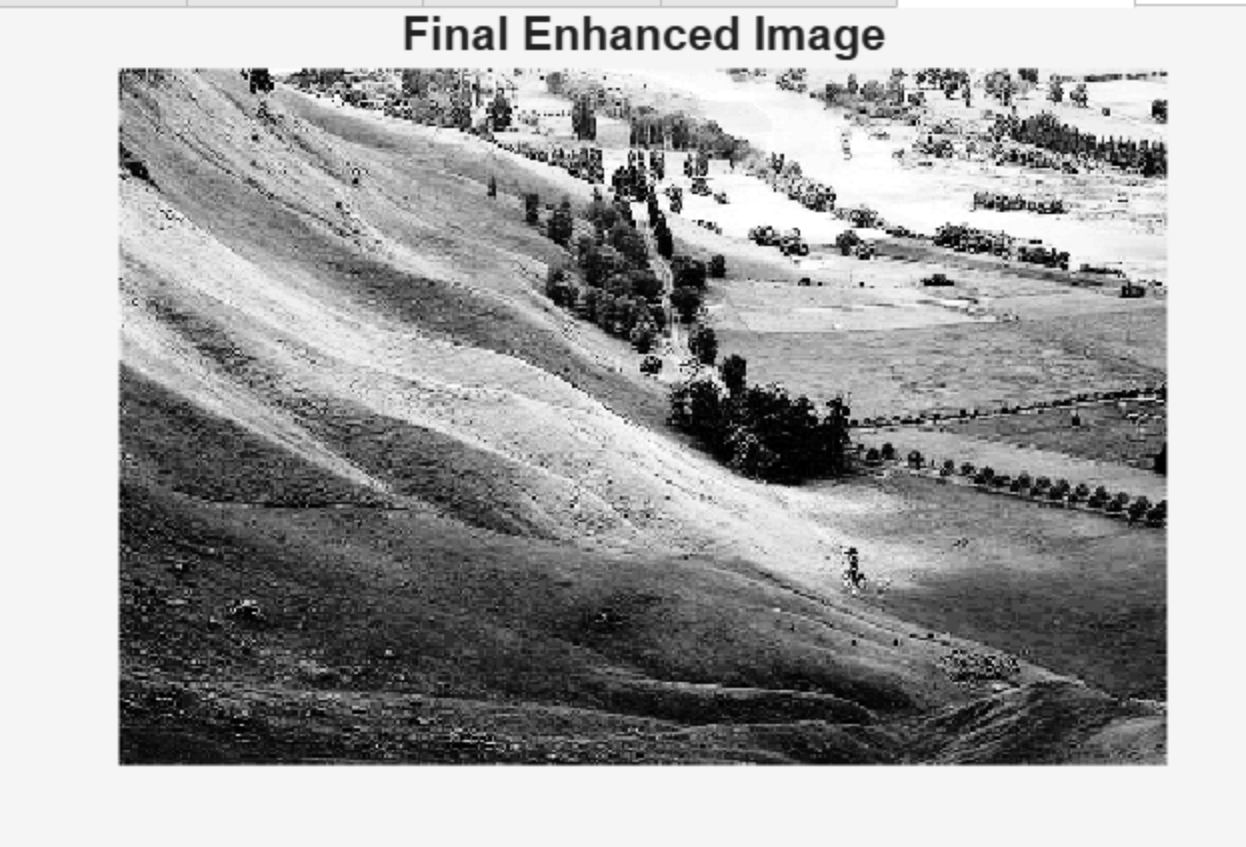
```
if size(img,3) == 3
    gray = rgb2gray(img);
else
    gray = img;
end

figure, imshow(gray), title('Grayscale Image');

%% Display Info
disp('Image Info:');
disp(size(gray));
disp(class(gray));
disp(gray(1:5,1:5));   % partial matrix

%% 2. Sampling (Resize)
scales = [0.5, 0.25, 1.5, 2];
for i = 1:length(scales)
    resized = imresize(gray, scales(i));
    figure, imshow(resized), title(['Scale = ', num2str(scales(i))]);
end

%% Quantization (Bit Depth Reduction)
levels = [256, 16, 4];  % 8-bit, 4-bit, 2-bit
for i = 1:length(levels)
    q_img = floor(double(gray)/(256/levels(i))) * (256/levels(i));
    q_img = uint8(q_img);
    figure, imshow(q_img), title(['Levels = ', num2str(levels(i))]);
end

%% 3. Geometric Transformations

% Rotation
angles = [30, 45, 90, 180];
for i = 1:length(angles)
    rot = imrotate(gray, angles(i), 'bilinear', 'crop');
    figure, imshow(rot), title(['Rotation ', num2str(angles(i))]);
end

% Translation
tform = affine2d([1 0 0; 0 1 0; 50 30 1]);
translated = imwarp(gray, tform);
figure, imshow(translated), title('Translated');

% Shearing
shear = affine2d([1 0.5 0; 0 1 0; 0 0 1]);
sheared = imwarp(gray, shear);
figure, imshow(sheared), title('Sheared');

%% 4. Intensity Transformations

% Negative
neg = 255 - gray;
figure, imshow(neg), title('Negative');

% Log Transform
```

```
log_img = uint8(255 * log(1 + double(gray)) / log(256));
figure, imshow(log_img), title('Log Transform');

% Gamma Correction
gamma1 = imadjust(gray, [], [], 0.5);
gamma2 = imadjust(gray, [], [], 1.5);

figure, imshow(gamma1), title('Gamma 0.5');
figure, imshow(gamma2), title('Gamma 1.5');

%% 5. Histogram Processing

figure;
imhist(gray);
title('Original Histogram');

eq_img = histeq(gray);

figure, imshow(eq_img), title('Histogram Equalized');
figure;
imhist(eq_img);
title('Equalized Histogram');

%% 6. Final Enhanced Image (Best Combination)

enhanced = process_image(img);

figure, imshow(enhanced), title('Final Enhanced Image');

imwrite(enhanced, 'output.jpg');
```

process_image.m:

```
function enhanced = process_image(input_image)

if size(input_image,3) == 3
    gray = rgb2gray(input_image);
else
    gray = input_image;
end

% Enhancement pipeline
enhanced = imadjust(gray, [], [], 0.5);
enhanced = histeq(enhanced);

end
```

Input image:



5. Conclusion

This project demonstrates how different image processing techniques can be combined to enhance image quality effectively. The integration of multiple methods results in a robust system capable of improving low-quality images for better visualization and analysis.

6. Questions & Answers

1. Why does histogram equalization improve contrast?

It spreads pixel intensities across full range → better visibility.

2. How does gamma affect brightness?

y < 1 : brighter
y > 1 : darker

3. What is the effect of quantization on image quality?

Lower bits = loss of detail + banding.

4. Which transformation is reversible and why?

Translation & rotation because info is not lost

5. How do transformations affect spatial structure?

Transformations change position/orientation of pixels

7. Tools Used

- MATLAB Online
- Image Processing Toolbox